

StatBot: 面向 AI + 统计的自主数据分析 Agent

摘要

本文介绍一个面向“AI + 统计”场景的智能数据分析 Agent 系统 StatBot。该系统由原始 LAMBDA 项目重构而来，目标是将自然语言需求、统计分析流程、代码执行环境与结果报告机制整合为一个可复现、可交互、可扩展的统计分析代理。围绕课程期末项目中对 Agent 系统的核心要求，本文重点展示了四类能力：自主决策与规划、工具调用与环境交互、记忆与上下文管理、多步骤任务执行。相比初始版本，StatBot 在三个方面进行了系统性增强：其一，引入结构化的统计工具注册表，将常见统计分析方法内置为可路由的技能集合；其二，引入会话级状态管理与浏览器会话隔离，避免不同分析任务之间的上下文污染；其三，引入模型不可达时的本地降级机制，使系统在外部大模型接口超时的情况下仍然能够完成代码执行、结果展示与报告生成。实验与功能测试表明，系统能够完成数据上传、数据概览、Notebook 导出、对话保存、报告生成、代码回显等前端交互，并能够在样例电商数据集上执行描述统计、假设检验、回归分析和可视化任务。该项目可视为一个面向统计分析教学、数据分析辅助和 AI4S 场景的通用型 Agent 原型。

1. 引言

1.1 项目背景与动机

在数据分析与统计建模 workflows 中，用户往往需要在多个工具之间切换：读取数据、进行变量检查、选择统计方法、编写分析代码、解释结果、导出报告。传统分析流程对用户的统计知识、编程能力与工具熟练度均有较高要求。随着大语言模型与代码代理技术的发展，构建一个能够理解自然语言需求、自动规划统计分析流程、调用执行环境并返回可解释输出的 Agent，成为“AI + 统计”方向中具有实际应用价值的研究与工程问题。

本项目面向《人工智能导论》期末项目的要求，选择“AI + 统计 / 数据分析 Agent”作为项目方向，尝试将大模型代理能力与经典统计方法相结合，构建一个具有实际可用性的统计分析 Agent 系统。

1.2 问题定义

本文所解决的问题可以表述为：

给定一个上传的数据集和用户的自然语言分析需求，设计一个能够自主规划分析步骤、选择合适统计方法、生成并执行代码、维护上下文、输出结构化分析结果与技术报告的 Agent 系统。

1.3 项目目标

本项目的目标包括：

1. 构建一个面向统计分析的交互式 Agent 系统；
2. 将常见统计方法封装为可路由的内置工具；
3. 引入可复现的代码执行环境与会话级上下文管理；
4. 设计在外部 LLM 网络异常时的降级策略，提升系统鲁棒性。

2. 相关工作

2.1 大模型 Agent 与任务执行

近年来，大模型 Agent 系统广泛用于自动化任务执行、工具调用和多步骤推理。典型工作强调通过“计划 -> 调用工具 -> 观察 -> 继续决策”的方式完成复杂任务。这类系统在编码辅助、办公自动化、网页操作和科研辅助中表现突出。

2.2 数据分析与 Auto-Analytics

数据分析 Agent 的核心难点在于：用户需求往往是自然语言表达，而真正的分析过程需要经过变量识别、方法选择、假设检查、结果解释与报告整合等多个步骤。若仅依赖自由生成代码，系统容易产生不稳定或不一致的分析行为。因此，将高频统计方法前置封装为结构化工具，是提升可控性和鲁棒性的有效路径。

2.3 本项目的改进点

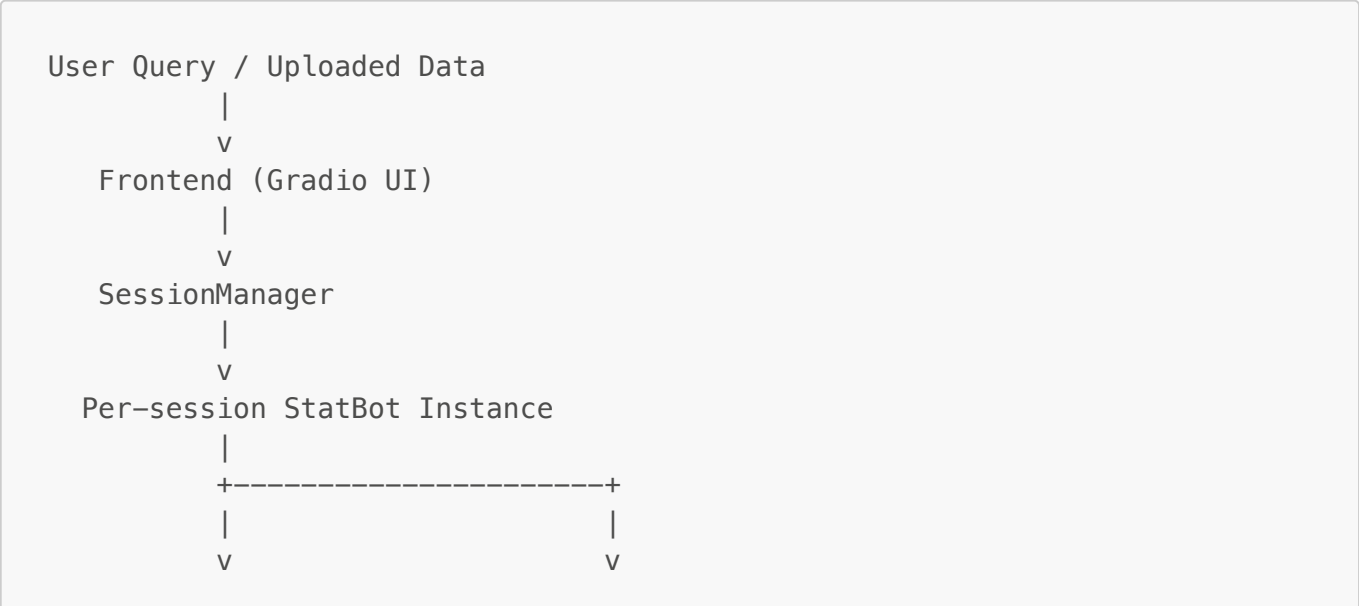
相较于原始版本和一般的“自由生成代码”式数据 Agent，本项目的主要创新点包括：

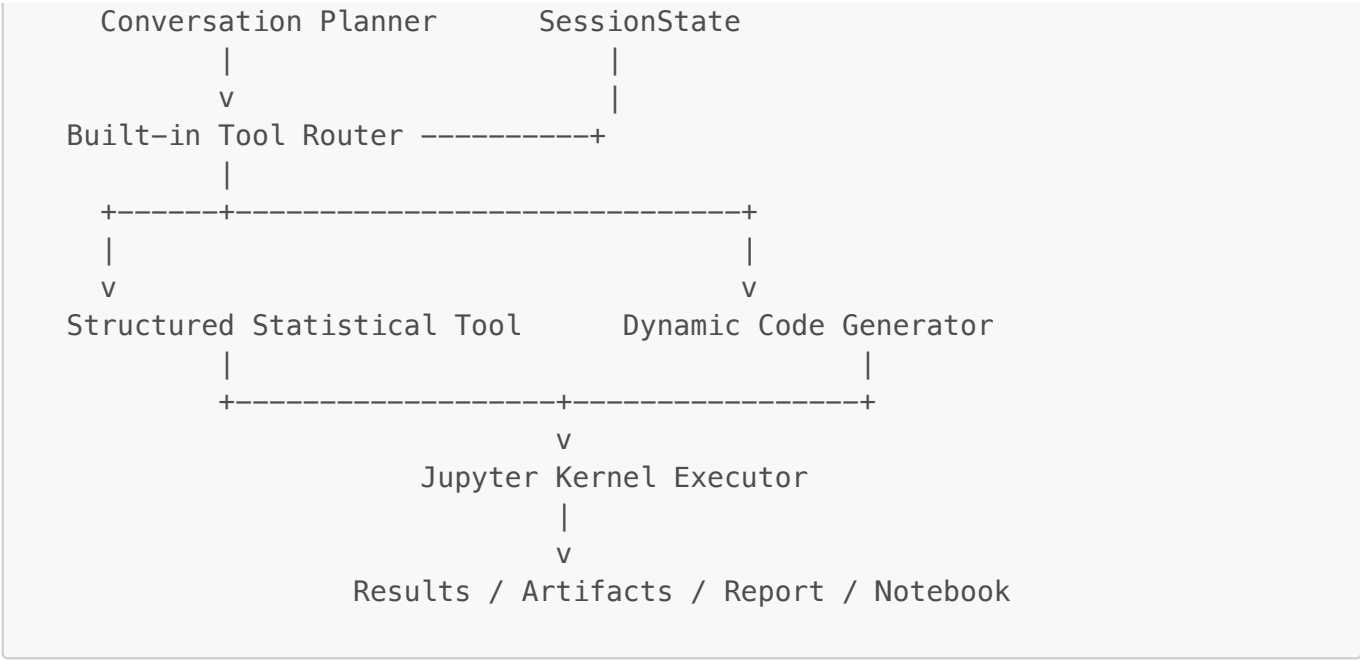
- 1. 统计工具注册表化
将常见统计方法封装为可匹配、可解释、可执行的内置工具，降低纯生成式代码的不稳定性。
- 2. 统计优先的分析工作流
在分析模式下引入显式 statistical planning，使 Agent 在生成代码前先构建统计分析计划。
- 3. 会话级上下文与状态持久化
用 `SessionState` 持久化记录上传文件、数据摘要、执行历史、工具选择、产物与错误状态。
- 4. 浏览器会话隔离机制
引入 `SessionManager`，使每个前端会话对应独立的 `StatBot` 实例，避免 `Edit Code` 与历史状态串扰。
- 5. 外部模型超时下的本地降级
在结果解释与报告生成阶段加入本地 fallback，使系统在 LLM 不可用时仍保持基本可用。

3. 方法设计

3.1 总体架构

`StatBot` 的总体架构如图所示（文本形式）：





系统由前端交互层、会话管理层、决策与工具路由层、代码执行层和结果呈现层组成。

3.2 Agent 架构设计

系统内部包含三个智能角色模块：

1. **Conversation 模块**

负责整体对话流程、统计规划、工具选择、执行结果整合和报告生成。

2. **Programmer 模块**

负责代码生成。当请求匹配内置统计工具时，Programmer 接收的是“工具嵌入代码”；否则退回动态代码生成。

3. **Inspector 模块**

在代码执行失败时提供错误诊断和修复建议，实现有限次的自修复循环。

3.3 统计工具与技能注册表

为了使 Agent 不再完全依赖自由生成代码，我们引入了 `builtin_skills.py` 中的结构化统计工具注册表。每个工具由以下字段描述：

- `name`：工具名
- `category`：类别
- `description`：功能说明
- `matcher`：匹配用户请求的规则
- `builder`：生成可执行代码片段的函数

该设计将“统计方法选择”从纯语言生成转化为“可解释的路由决策 + 模板化代码执行”，增强了系统可控性。

3.4 内置统计方法覆盖范围

当前系统已内置以下分析能力：

3.4.1 描述统计与数据质量

- 数据概览
- 描述统计
- 缺失值分析
- 异常值筛查
- 分组统计
- 类别频数统计
- 列联表汇总

3.4.2 假设检验

- 单样本 t 检验
- 双样本 t 检验
- 配对 t 检验
- Wilcoxon 符号秩检验
- Mann-Whitney U 检验
- 单因素 ANOVA
- Kruskal-Wallis 检验
- 卡方检验
- Fisher 精确检验
- 双比例检验
- 正态性检验
- 方差齐性检验

3.4.3 回归与关系分析

- Pearson / Spearman 相关分析
- OLS 线性回归
- Logistic 回归
- 散点回归图

3.4.4 可视化

- 分布图
- 时间趋势图
- 分组箱线图
- Pairplot

3.5 上下文管理与记忆机制

系统通过 **SessionState** 维护会话记忆，记录内容包括：

- 上传文件与数据摘要
- 最近用户请求
- 最近完成的工作摘要
- 当前阶段状态
- 最近选中的工具及其假设
- 最近成功执行的代码
- 最近执行结果摘要

- 最近错误信息
- 产物列表（图像、文件、报告、Notebook）

这使得系统具备“可回溯的上下文管理能力”，满足课程项目中对记忆与上下文管理的要求。

3.6 工具调用与环境交互设计

本项目的“工具调用”并非仅指 LLM function-calling，而是指 Agent 对外部环境能力的调用与协调，具体包括：

1. 数据文件上传与缓存目录写入
2. Jupyter Kernel 启动与 Python 代码执行
3. 读取 / 更新 DataFrame
4. 自动生成 Notebook
5. 自动生成 Markdown 报告
6. 检测与展示图片、文件等分析产物

这构成了一个“统计分析工作台式 Agent”，而不仅是一个聊天机器人。

4. 实现细节

4.1 关键模块实现

4.1.1 `builtin_skills.py`

该模块实现了结构化统计工具注册表。用户请求会被标准化后送入一组 `matcher` 规则，若命中则返回带有：

- 工具名
- 类别
- 选择理由
- 统计假设
- 可执行代码

的工具对象，并通过 `embed_skill_code()` 注入给代码执行链路。

4.1.2 `conversation.py`

该模块是核心编排器，主要负责：

- 统计计划生成
- 会话状态更新
- 工具选择
- 动态代码生成或工具代码嵌入
- 执行结果处理
- 报告生成
- 错误与降级逻辑

4.1.3 `session_manager.py`

为解决原系统中不同前端会话共享同一个 `StatBot` 实例的问题，本项目新增 `SessionManager`：

- 以 `request.session_hash` 为键
- 为每个前端会话分配独立的 `StatBot`
- 使用 LRU 风格淘汰机制限制会话数量

这一修改直接修复了 `Edit Code` 按钮读到其他会话旧代码的问题。

4.2 本次关键修复

4.2.1 修复 `Edit Code` 串会话问题

原始问题：

- 所有浏览器会话共享同一个 `StatBot` 实例
- `rendering_code()` 从全局消息历史中寻找最近一次代码
- 导致新会话可能读到旧会话代码

修复方式：

- 在 `statbot_app.py` 中使用 `SessionManager`
- 所有按钮回调均通过 `request.session_hash` 路由到当前会话实例
- 无代码时返回明确提示：`No code has been generated in this session yet.`

4.2.2 修复 `Submit` 在模型超时下仅报错的问题

原始问题：

- 分析代码可能已经执行成功
- 但最后一步“模型解释结果”超时，会让用户误以为整个分析失败

修复方式：

- 在 `_handle_execution_result()` 中捕获解释阶段超时
- 若超时，则调用 `_fallback_result_response()`
- 返回本地摘要、技术说明和后续建议

这样即使外部模型不可用，前端仍会显示：

- 执行结果
- 本地 fallback summary
- 标准格式的 next-step 建议

4.2.3 修复 `Generate Report` 在模型超时下完全失败的问题

原始问题：

- 报告生成完全依赖外部 LLM
- 一旦网络超时，报告按钮直接失败

修复方式：

- 在 `document_generation()` 中加入本地报告生成逻辑
- 使用 `SessionState`、对话历史、执行摘要和产物列表构建 deterministic markdown report

- 在模型不可达时自动回退到 `_build_local_report()`

4.3 技术难点与解决方案

技术难点	原因	解决方案
会话状态污染	单例式后端共享状态	引入 <code>SessionManager</code> 做会话隔离
上传对象兼容性	Gradio 新旧版本文件对象格式不同	兼容 <code>str</code> / 文件对象 / 列表三种输入形态
结果解释阶段脆弱	模型依赖强、网络超时频繁	本地 <code>fallback summary</code>
报告生成易失败	完全依赖大模型	本地 <code>fallback report</code>
统计方法选择不稳定	自由生成代码可控性差	结构化统计工具路由

5. 实验与结果

5.1 实验设置

实验环境如下：

- Python 3.10.19
- Gradio 4.44.1
- Jupyter Kernel (`python3`)
- 前端本地部署于 `127.0.0.1`
- 示例数据：`demo_data/ecommerce_demo.csv`

系统运行模式为统计优先模式：

- `analysis_profile = statistics`
- `statistical_planning = True`

5.2 功能验证

我们对前端主要按钮进行了真实测试。由于本机 `Computer Use` 插件版本不匹配，无法直接接管本机浏览器，因此采用两种真实测试方式：

- 直接调用正在运行的 Gradio 按钮 API；
- 直接调用与按钮绑定的同一套后端函数。

测试结果如下：

功能	结果	说明
Upload Data	正常	正确返回文件名、行数、列数
Show/Update DataFrame	正常	上传后可展示 <code>48 x 17</code> 数据表
Notebook	正常	可导出 <code>notebook.ipynb</code>
Save Dialogue	正常	可写出对话与配置文件
Clear All	正常	可清空状态并重启 kernel

功能	结果	说明
Edit Code	修复后正常	不再读到其他会话的历史代码
Submit	修复后可降级	分析执行成功时可输出本地 fallback summary
Generate Report	修复后可降级	LLM 超时时仍可生成本地 report.md

5.3 案例展示

在样例电商数据集上，系统可以自动完成如下任务：

- 1. 读取数据集，识别共有 48 行、17 列；
- 2. 生成数据概览与缺失值汇总；
- 3. 根据用户问题自动命中内置统计工具；
- 4. 在 notebook kernel 中执行结构化统计代码；
- 5. 输出数值摘要、类别摘要、预览表格等结果；
- 6. 在解释阶段超时时，提供本地摘要和后续建议。

一个典型的降级后回复由三部分组成：

- 统计分析计划
- 执行结果面板
- 本地 fallback summary + next steps

5.4 结果分析

实验表明，本项目并不把“外部大模型能否联网”视为系统唯一成败条件。相反，系统通过工具注册、上下文管理与本地降级设计，使得以下能力可以在模型不稳定时仍然保留：

- 数据读取与检查
- 统计方法执行
- 代码产物保存
- Notebook 导出
- 本地报告生成

这说明 StatBot 从“单纯依赖模型生成内容”演化为“模型增强的、具备执行内核和可回退路径的统计分析 Agent”。

6. 与课程考核要求的对应关系

根据《人工智能导论期末项目考核要求》，项目需体现 Agent 的核心特性。StatBot 的对应关系如下：

6.1 自主决策与规划能力

- 使用 STAT_PLANNER_PROMPT 构建统计分析计划
- 根据用户需求与数据特征选择工具或动态代码路径
- 在多种统计方法之间做条件化选择

6.2 工具调用与环境交互能力

- 调用文件上传与缓存管理
- 调用 Jupyter Kernel 执行 Python 代码
- 调用 DataFrame 展示、Notebook 导出、报告生成等功能

6.3 记忆与上下文管理能力

- **SessionState** 记录数据摘要、请求历史、工具历史、执行结果和产物
- **SessionManager** 保证会话隔离，避免跨任务污染

6.4 多步骤任务执行能力

系统完整 workflow 包括：

1. 接收请求
2. 生成统计计划
3. 选择工具或生成代码
4. 执行代码
5. 解释结果
6. 记录产物
7. 生成报告

因此，该系统与课程对 Agent 项目的要求高度一致。

7. 总结与展望

本文围绕“AI + 统计”场景，设计并实现了一个面向智能数据分析的 Agent 系统 **StatBot**。与原始版本相比，系统在统计方法覆盖、上下文管理、前端会话隔离、异常降级与结果可复现性等方面均得到了明显增强。项目不仅能够自动执行多种常见统计分析任务，而且能够在外部模型超时情况下保持基本可用性，从而具备更强的工程稳定性。

系统目前仍存在一些局限：

1. 复杂开放式问题仍可能退回动态代码生成，稳定性不如内置工具；
2. 本地 fallback summary 的解释深度不及完整大模型生成；
3. 浏览器级自动化测试能力仍受本机插件版本影响；
4. 统计报告的视觉表现仍可进一步增强。

未来工作包括：

1. 扩展更多统计方法，如生存分析、时间序列建模、因果推断和贝叶斯分析；
2. 增加前端可视化工具面板，实现“点选式统计分析”；
3. 引入结构化 artifact registry 和长对话摘要机制；
4. 增加测试集与 benchmark，量化 Agent 的任务成功率与统计正确性；
5. 导出正式 PDF 报告、PPT 与 Poster 资产，形成完整课程项目交付。

8. 参考文献

1. McKinney, W. Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 2010.

2. Pedregosa, F., et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 2011.
 3. Seabold, S., and Perktold, J. Statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of the 9th Python in Science Conference*, 2010.
 4. Virtanen, P., et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 2020.
 5. Hunter, J. D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 2007.
 6. Perez, F., and Granger, B. E. IPython: A System for Interactive Scientific Computing. *Computing in Science & Engineering*, 2007.
 7. Yao, S., et al. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022.
 8. Gradio Team. Gradio Documentation. <https://www.gradio.app/>
-

9. 附录：当前项目改造摘要

本次针对 **StatBot** 的改造可以概括为：

1. 引入会话记忆模块 **SessionState**；
2. 引入统计工具注册表 **builtin_skills.py**；
3. 扩展描述统计、假设检验、回归、可视化等常见方法；
4. 修复上传无反馈、前端空转不报错等问题；
5. 修复 **Edit Code** 串会话问题；
6. 为 **Submit** 和 **Generate Report** 增加本地降级路径；
7. 增加样例数据 **ecommerce_demo.csv** 以支持演示与案例展示。